Распределённые приложения: пулы, JTA и 2PC

Информационные системы · Конспект лекции

JDBC и пул соединений

Создание соединения с БД дорого и ограничено ресурсами. Если каждый клиент открывает его отдельно, растут задержки и нагрузка. Connection pool повторно выдаёт ограниченный набор соединений; приложению обычно достаточно получить DataSource (например, через JNDI), взять соединение и закрыть его после работы — закрытие возвращает соединение в пул.

JPA и типы транзакций

Persistence Unit задаёт конфигурацию ORM, EntityManagerFactory создаёт EntityManager, а EntityManager управляет persistence context. Resource-local транзакцией управляют через API провайдера, обычно для одного ресурса. JTA координирует транзакции контейнера и может охватывать несколько ресурсов. Правильный тип должен соответствовать конфигурации persistence unit и транзакционного менеджера.

CMT и BMT

При Container-Managed Transactions контейнер начинает/завершает транзакцию согласно атрибуту метода. REQUIRED присоединяется к текущей или создаёт новую; REQUIRES_NEW приостанавливает текущую и запускает отдельную; MANDATORY требует существующую; SUPPORTS работает с ней или без неё; NOT_SUPPORTED временно приостанавливает; NEVER запрещает вызов внутри транзакции. BMT позволяет компоненту явно вызывать begin/commit/rollback, но увеличивает ответственность за все пути завершения.

Распределённая транзакция

Если одна бизнес-операция меняет несколько ресурсов, например две БД, может потребоваться атомарно зафиксировать изменения везде. JTA предоставляет API координации, а XA — протокол взаимодействия менеджера транзакций с ресурсами. Участники должны поддерживать подготовку и фиксацию; не каждый ресурс или драйвер поддерживает XA.

Двухфазная фиксация (2PC)

Координатор в фазе Prepare запрашивает у каждого участника готовность зафиксировать изменения. Участник записывает нужное состояние и отвечает Yes/No. Если все готовы, координатор рассылает Commit; иначе — Abort. Участник подготовленной транзакции удерживает ресурсы до решения. Журналы позволяют восстановить решение после сбоя.

Цена 2PC и отказоустойчивость

2PC сохраняет атомарность при участии совместимых менеджеров, но может блокироваться при отказе координатора после Prepare. Удержание блокировок и сетевые задержки снижают доступность. Нужны тайм-ауты, устойчивые журналы, повторная передача решения и восстановление. Тайм-аут сам по себе не заменяет согласованное решение: участник не всегда может безопасно выбрать commit/abort самостоятельно.

Альтернативы и границы

Не каждая распределённая операция оправдывает XA. Для долгих бизнес-процессов часто применяют Saga: последовательность локальных транзакций и компенсирующие действия. Outbox помогает надёжно публиковать событие вместе с локальной фиксацией. Эти подходы дают eventual consistency, а не глобальную атомарность, и требуют идемпотентности и обработки повторов.

Spring и JTA

Spring предоставляет абстракцию управления транзакциями и декларативную аннотацию @Transactional; конкретный менеджер выбирается по окружению — локальный или JTA. Атрибуты propagation задают присоединение/создание транзакции, isolation — видимость изменений, rollback rules — откат при исключениях. Важно помнить о прокси-вызовах и внутреннем вызове метода: аннотация срабатывает только когда запрос проходит через механизм перехвата.

Что запомнить

Пулы переиспользуют соединения, JTA координирует транзакции контейнера, а 2PC обеспечивает согласованную фиксацию совместимых участников ценой блокировок и сложности восстановления. Выбирайте локальную транзакцию, XA или Saga исходя из требований к атомарности и доступности.

Термины

DataSource · пул соединений · JTA · XA · 2PC · Saga · Outbox

Составлено по материалам исходной лекции. Проверьте поколение API и особенности используемой СУБД/контейнера.